



KIIT Deemed to be University

School of Computer Engineering

Bhubaneswar, Odisha - 751024

# A PROJECT REPORT

on

Medical Tourism Platform In Bhubaneswar

Submitted in partial fulfilment of the requirements for the  
award of a Bachelor's Degree in Computer Science

By

|                       |          |
|-----------------------|----------|
| Akash Chandra         | 22051659 |
| Kaushal Kumar         | 22051687 |
| Shilky Dwivedi        | 22052058 |
| Namitha Mohan         | 2206415  |
| Amisha Tarun Dewangan | 2229204  |
| Pranjal Mohanty       | 2229137  |

Under the Guidance of

**Pr. Amit Kumar**

# Certificate

This is to certify that the project entitled “Medical Tourism Platform” submitted by Akash Chandra and group is a record of original and bona fide work carried out in partial fulfilment of the requirement for the award of the Degree of Bachelor of Technology in Information Technology at KIIT Deemed to be University, Bhubaneswar.

The work embodied in this report has been carried out under our supervision during the academic year 2025–2026. The project work presented here is the result of the sincere effort, dedication, and commitment of the students. It demonstrates their ability to apply theoretical knowledge to practical problems and reflects their competence in software engineering, web development, and database integration.

We hereby declare that this project report has not been submitted to any other University or Institute for the award of any degree or diploma. The contents of this report are original to the best of our knowledge, and the students have adhered to academic integrity and ethical standards throughout the course of this work.

This certificate is issued as a testimony to the successful completion of the project work under our guidance. We commend the students for their innovative approach, systematic methodology, and perseverance in completing the project within the stipulated time frame. Their work contributes to the academic environment of the School of Computer Engineering and serves as an example for future batches.

**Pr. Amit Kumar**

Project Guide

School of Computer Engineering

**Place: Bhubaneswar**

# Acknowledgements

We express our sincere gratitude to **Pr. Amit Kumar , KIIT Deemed to be University, Bhubaneswar**, for their invaluable guidance, technical insights, and continuous support throughout this project. Their expertise and constructive feedback have been instrumental in refining the research and achieving meaningful outcomes.

We extend our appreciation to KIIT Deemed to be University, Bhubaneswar, for providing the necessary infrastructure, computational resources and academic environment that facilitated the successful execution of this project.

We also acknowledge the contributions of our peers and colleagues for their insightful discussions, collaborative efforts, and critical feedback, which significantly enhanced the depth of this study.

Akash Chandra  
Shilky Dwivedi  
Kaushal Kumar  
Namitha Mohan  
Amisha Tarun Dewangan  
Pranjal Mohanty

# Abstract

The healthcare industry is rapidly evolving, and with the rise of medical tourism, patients often face challenges in identifying suitable doctors across different hospitals. Existing platforms provide fragmented information, which makes it difficult for patients to compare doctors based on specialization, experience, and hospital affiliation. This project addresses that gap by developing a Medical Dashboard that enables hospital-wise doctor search with department-based filtering. The system is designed to be user-friendly, visually engaging, and technically robust, ensuring that patients can access accurate and structured information with minimal effort.

The dashboard integrates a Spring Boot backend to handle RESTful APIs, a MongoDB database for storing hospital and doctor records, and a React frontend for interactive user interfaces. Hospitals are represented as cards with logos, and each hospital provides a list of doctors displayed in profile-style cards. Patients can refine their search using department filters, making it easier to locate specialists such as cardiologists, neurologists, or pathologists. The system also supports navigation between hospitals, ensuring flexibility and ease of use.

From a technical perspective, the project demonstrates the integration of multiple technologies into a cohesive system. The backend ensures scalability and secure data handling, while the frontend emphasises responsiveness and modular design. Testing was conducted using Postman for API validation and manual verification of frontend navigation. The results confirmed that the dashboard successfully retrieves and displays doctors hospital-wise, with accurate filtering by department. This validates the system's effectiveness in bridging the gap between patients and healthcare providers.

The project contributes to the broader vision of digital healthcare by offering a structured solution for hospital-wide doctor discovery. It highlights the importance of combining software engineering principles with practical healthcare needs. The dashboard not only serves as a prototype for medical tourism platforms but also lays the foundation for future enhancements such as appointment booking, patient reviews, and integration with payment gateways. By addressing a real-world problem with a practical solution, this project demonstrates the application of academic.

# Content

|   |                                                |                                        |    |
|---|------------------------------------------------|----------------------------------------|----|
| 1 | Introduction                                   |                                        | 8  |
| 2 | Basic Concepts and Study of the Subjects       |                                        | 9  |
|   | 2.1                                            | Web Development                        | 10 |
|   | 2.2                                            | Frontend Development (React)           | 10 |
|   | 2.3                                            | Backend Development (Spring Boot)      | 11 |
|   | 2.4                                            | Database Management (MongoDB)          | 12 |
|   | 2.5                                            | API Integration                        |    |
| 3 | Problem Statement / Requirement Specifications |                                        | 13 |
|   | 3.1                                            | Project Statement                      | 13 |
|   | 3.2                                            | Project Planning                       | 13 |
|   | 3.3                                            | Requirement Specification              |    |
|   | 3.4                                            | System Design                          |    |
|   | 3.5                                            | System Architecture(UML/Block Diagram) | 17 |
| 4 | Implementation                                 |                                        | 18 |
|   | 4.1                                            | Methodology / Proposal                 | 18 |
|   | 4.2                                            | Testing / Verification Plan            | 24 |

|                         |                             |                                      |               |
|-------------------------|-----------------------------|--------------------------------------|---------------|
|                         | 4.3                         | Result Analysis / Screenshots        | 25            |
|                         | 4.4                         | Quality Assurance                    | 28            |
| 5                       | Standard Adopted            |                                      | 29            |
|                         | 5.1                         | Design Standards                     |               |
|                         | 5.2                         | Coding Standards                     |               |
|                         | 5.3                         | Testing Standards                    |               |
|                         | 5.4                         | Ethical and Medical Safety Standards |               |
| 6                       | Conclusion and Future Scope |                                      | 30            |
|                         | 6.1                         | Conclusion                           | 30            |
|                         | 6.2                         | Future Scope                         | 31            |
| References              |                             |                                      | 32            |
| Individual Contribution |                             |                                      | 33<br>-4<br>2 |

# Chapter-1

## INTRODUCTION

The healthcare sector is getting more into using solutions to make things easier for people to get the medical help they need. Most of the time the information about doctors and hospitals is all over the place and not organised, which makes it really tough for patients to find the right doctor. This is a problem, especially when people are travelling to another country for medical care, because they need to be able to trust the information they are getting so they can make good choices.

The Medical Tourism Platform was made to solve this problem. It is a system that lets people look for doctors based on the hospital they work at and what kind of medicine they practice. The platform is easy to use. It shows hospitals as cards that you can click on, and each hospital has a lot of information about the doctors who work there. This makes it a lot simpler for people to find the help they need.

### 1.1 Project Overview

The Medical Tourism Platform is a web-based application designed to help patients easily discover doctors across different hospitals. The system integrates a frontend built using React, a backend developed with Spring Boot, and a MongoDB database for storing hospital and doctor information.

The platform allows users to:

- View hospitals in a card-based layout
- Access a list of doctors associated with each hospital
- Filter doctors based on departments such as cardiology, neurology, etc.

The system follows a three-tier architecture, ensuring smooth interaction between the user interface, backend services, and database. It is designed to be scalable, responsive, and easy to use, providing a seamless experience for users.

### 1.2 Aim of the Project

The main aim of this project is to develop a centralized and user-friendly medical dashboard that enables patients to search for doctors hospital-wise and filter them based on their specialisation, thereby improving accessibility and decision-making in healthcare services.

### **1.3 Objectives of the Project**

The key objectives of the project are:

1. To design an interactive interface that displays hospitals and doctors in a structured format.
2. To implement hospital-wise doctor search functionality.
3. To provide department-based filtering for easy identification of specialists.
4. To develop a robust backend using Spring Boot for handling API requests.
5. To use MongoDB for efficient storage and management of healthcare data.
6. To ensure smooth integration between frontend, backend, and database.
7. To create a responsive and user-friendly platform for better user experience.

# CHAPTER-2

## Basic Concepts and Study of Subjects

The Medical Tourism Platform brings together parts of modern web development. These include frontend engineering, backend systems, database management and API-based communication. The goal is to create a healthcare information system that helps users find hospitals and doctors. They can filter doctors by their speciality. This chapter explains the theory, technologies and concepts behind the system. It covers user interface design, RESTful API development, database handling and system integration.

### 2.1 Web Development Fundamentals

Web development is the backbone of the system. It makes the interface interactive and user-friendly. Web development involves structuring, styling and adding functionality to web pages.

#### A. HTML (HyperText Markup Language):-

HTML defines the structure of the application. It provides the layout for hospital cards, doctor profiles, navigation bars and filtering sections. HTML ensures the organisation of content.

#### B. CSS (Cascading Style Sheets)

CSS controls the presentation of the application. It handles layout design, colours, fonts, spacing and responsiveness. CSS makes the dashboard visually appealing and adaptable to screen sizes.

#### C. JavaScript

JavaScript adds interactivity and dynamic behaviour to the application. It handles user events like clicking, filtering and navigation. JavaScript enables dynamic content updates without reloading the page.

### 2.2 Frontend Development using React

React is a JavaScript library. It builds the frontend of the application. React creates fast and scalable user interfaces.

#### A. Component-Based Architecture

React divides the UI into components. These include HospitalList, DoctorList and DashboardPage. This approach improves maintainability, reusability and scalability.

#### B. State Management and Rendering

React uses state management to handle data. The virtual DOM ensures rendering. It updates the required parts of the UI.

### C. Application in This Project

- \* Displaying hospital cards dynamically
- \* Rendering doctor lists based on selected hospital
- \* Implementing department-based filtering
- \* Managing navigation between pages

## **2.3 Backend Development using Spring Boot**

The backend is developed using Spring Boot. It simplifies Java-based web application development. Supports RESTful services.

### A. RESTful API Development

Spring Boot creates APIs that handle communication between the frontend and the database. These APIs manage requests like fetching hospital data and retrieving doctor lists.

### B. Layered Architecture

The backend follows an architecture. It includes the Controller Layer, Service Layer and Repository Layer.

### C. Advantages

- \* Scalability and performance
- \* Secure data handling
- \* integration with frontend and database

## **2.4 Database Management using MongoDB**

MongoDB is a NoSQL database. It manages application data in a flexible format.

### A. Data Storage Model

MongoDB stores data in JSON documents. This allows flexible schema design. It's useful for handling varying data like doctor details and hospital information.

### B. Collections Used

- \* Hospital Collection → Stores hospital details
- \* Doctor Collection → Stores doctor information

### C. Advantages

- \* Flexible schema design
- \* Fast data retrieval
- \* Easy integration with Spring Boot

## 2.5 API Integration and Data Flow

API integration connects the frontend and backend components.

### A. Working Mechanism

1. User performs an action (e.g. selects a hospital)
2. Frontend sends request to backend API.
3. Backend processes. Retrieves data from MongoDB.
4. Response is sent back to frontend.

### B. Importance

- \* Enables real-time data updates
- \* Ensures communication between system components
- \* Improves performance and user experience

## 2.6 Testing using Postman

Testing ensures the correctness and reliability of the system.

### A. API Testing

Postman tests backend APIs by sending requests and verifying responses.

### B. Validation

- \* Ensures data retrieval
- \* Verifies filtering functionality
- \* Checks API performance and response time

## 2.7 Domain Knowledge: Healthcare and Medical Tourism

The project focuses on tourism. Patients face challenges in finding doctors based on hospital affiliation and specialisation.

### A. Problems Addressed

- \* Lack of hospital-wise doctor information
- \* Difficulty in finding specialists
- \* Time-consuming search process

## **B. Solution Provided**

The platform offers:

- \* Organized hospital listings
- \* Doctor profiles with detailed information
- \* Department-based filtering

## **C. Impact**

- \* Improves accessibility to healthcare services
- \* Helps patients make informed decisions
- \* Enhances overall user experience

The Medical Tourism Platform uses frontend technologies like React, backend frameworks like Spring Boot and database systems, like MongoDB. It also uses API communication and domain knowledge.

This ensures that the platform is scalable, efficient and capable of solving real-world healthcare challenges.

## 3. Problem Statement / Requirement Specifications

The healthcare industry faces significant challenges in providing structured and easily accessible information about doctors and hospitals. Patients often struggle to find suitable doctors based on hospital affiliation and specialization due to scattered and unorganized data across different platforms. This issue becomes more critical in medical tourism, where patients need reliable and well-organized information to make informed decisions.

The proposed **Medical Tourism Platform** addresses this problem by providing a centralized dashboard that allows users to search for doctors hospital-wise and filter them based on departments. The system ensures a user-friendly interface, efficient data handling, and seamless interaction between frontend, backend, and database components.

### 3.1 Project Statement

The objective of this project is to design and develop a web-based Medical Tourism Platform that enables users to search for doctors based on hospital and specialization. The system provides a structured and interactive interface where hospitals are displayed as cards, and each hospital contains a list of associated doctors.

The platform integrates frontend technologies with backend APIs and a database to ensure efficient data retrieval and display. It aims to simplify the process of finding doctors and improve accessibility to healthcare services.

### 3.2 Project Planning

The development of the Medical Tourism Platform follows a structured and phased approach to ensure systematic implementation and integration of all components.

#### Project Phases and Timeline:

##### Phase 1 – Requirement Analysis

- Understanding user needs and healthcare challenges
- Identifying system features and functionalities
- Defining project scope

##### Phase 2 – System Design

- Designing system architecture
- Creating UI layouts (hospital cards, doctor list)
- Planning database schema

##### Phase 3 – Backend Development

Developing RESTful APIs using Spring Boot

- Implementing controller, service, and repository layers
- Connecting backend with MongoDB

### **Phase 4 – Database Development**

- Creating collections for hospitals and doctors
- Storing structured data (specialization, experience, etc.)

### **Phase 5 – Frontend Development**

- Building UI using React
- Creating reusable components (HospitalList, DoctorList)
- Implementing filtering functionality

### **Phase 6 – Integration**

- Connecting frontend with backend APIs
- Ensuring smooth data flow

### **Phase 7 – Testing and Validation**

- Testing APIs using Postman
- Verifying UI functionality
- Fixing bugs and improving performance

### **Phase 8 – Deployment & Documentation**

- Preparing project report
- Final testing and demonstration

## **3.3 Requirement Specification**

### **A. Functional Requirements**

- The system should display hospitals in a card-based layout.
- The system should allow users to view doctors associated with a selected hospital.
- The system should provide department-based filtering of doctors.
- The system should fetch data dynamically from the backend.
- The system should display doctor details such as specialization, experience, and fees.

### **B. Non-Functional Requirements**

- The system should be user-friendly and easy to navigate.
- The system should be responsive across different devices.
- The system should provide fast data retrieval and display.
- The system should ensure data consistency and reliability.
- The system should be scalable for future enhancements.

### **C. Hardware Requirements**

- Basic computer/laptop
- Internet connection

### **D. Software Requirements**

- Frontend: React, HTML, CSS, JavaScript
- Backend: Spring Boot
- Database: MongoDB
- Testing Tool: Postman
- Development Tools: VS Code / IntelliJ

## **3.4 System Design**

The system is designed using a modular and layered approach to ensure flexibility and scalability. Each component of the system is developed independently and integrated to form a complete application.

### **• Input Layer**

User interacts with the system by selecting hospitals and applying filters.

### **• Processing Layer**

- React handles UI logic and user interaction
- Spring Boot processes API requests
- Business logic is handled in service layer

### **• Storage Layer**

- MongoDB stores hospital and doctor data
- Data is structured in collections

### **• Output Layer**

- Displays hospital cards
- Shows doctor profiles
- Displays filtered results

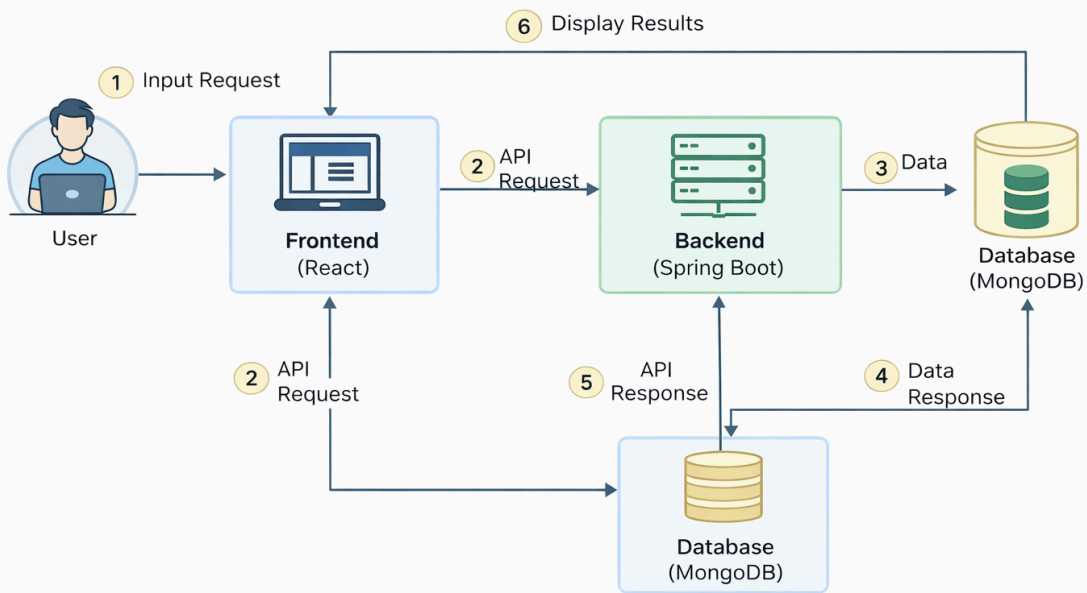


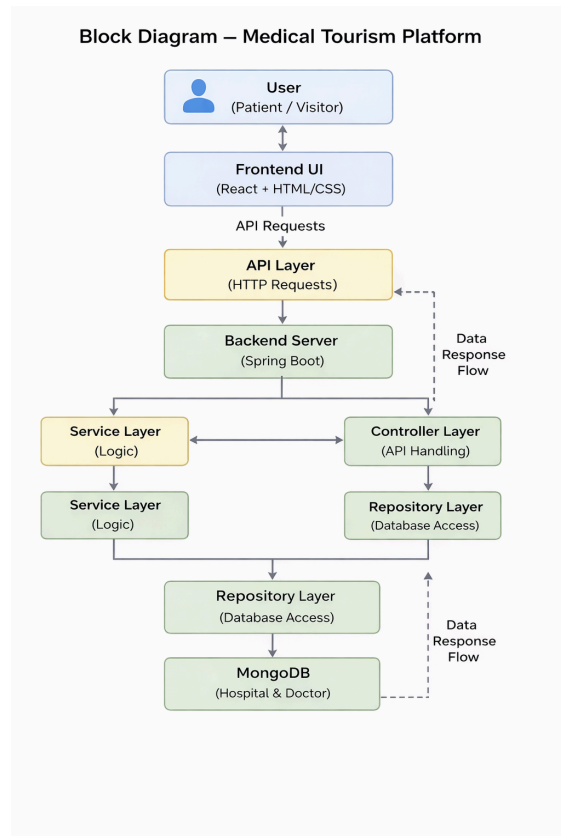
Fig 2.X System Workflow of Medical Tourism Platform

### 3.5 System Architecture (UML / Block Diagram)

The system follows a **three-tier architecture**, consisting of frontend, backend, and database layers.

#### Architecture Description:

1. **Frontend Layer (React)**
  - Handles user interface
  - Displays hospitals and doctors
  - Sends API requests
2. **Backend Layer (Spring Boot)**
  - Processes requests
  - Contains business logic
  - Provides RESTful APIs
3. **Database Layer (MongoDB)**
  - Stores hospital and doctor data
  - Retrieves data based on queries



The system architecture ensures clear separation of concerns, efficient data flow, and scalability, making the Medical Tourism Platform a reliable and user-friendly healthcare solution.

# CHAPTER 4

## IMPLEMENTATION

This chapter describes the methodologies, system workflow, and technical implementation strategies used in developing the **Medical Tourism Platform**. The system integrates frontend technologies, backend APIs, and database management into a unified healthcare information system.

The platform is designed using a modular approach, where each component—hospital display, doctor listing, filtering system, and API integration—is implemented independently to ensure scalability, maintainability, and efficient performance.

### 4.1 Methodology / Proposal

The system is implemented using a **three-tier architecture**, combining frontend, backend, and database technologies. The implementation is divided into key modules:

#### 4.1.1 Hospital Display Module Implementation

##### A. Data Preparation

Hospital data is stored in MongoDB with fields such as:

- Hospital name
- Location
- Logo/image
- Associated doctors

##### B. Data Storage

Hospitals are stored in a **MongoDB collection** in JSON format for flexibility.

##### C. API Development

Spring Boot APIs are created to fetch hospital data:

- GET /hospitals → Fetch all hospitals

##### D. Frontend Rendering

- React fetches hospital data using API
- Displays hospitals as **interactive cards**

##### E. Outputs

- List of hospitals
- Clickable hospital cards

#### 4.1.2 Doctor Listing Module Implementation

## **A. Data Structure**

Doctor data includes:

- Name
- Specialization
- Experience
- Consultation fee
- Hospital ID

## **B. Backend Processing**

- API retrieves doctors based on the selected hospital
- Endpoint example:  
GET /doctors?hospitalId=123

## **C. Frontend Logic**

- React displays doctor profiles in card format
- Dynamic rendering based on selected hospital

## **D. Outputs**

- List of doctors for selected hospital
- Profile-style doctor cards

## **4.1.3 Department Filtering Implementation**

### **A. Filtering Logic**

Filtering is implemented based on doctor specialization.

### **B. Process Flow**

1. User selects a department (e.g., Cardiology)
2. Filter condition is applied in frontend/backend
3. Only matching doctors are displayed

### **C. Implementation Approach**

- JavaScript filtering (frontend) OR
- API-based filtering (backend)

### **D. Outputs**

- Filtered doctor list
- Real-time UI updates

## **4.1.4 Backend Implementation (Spring Boot)**

### **A. Architecture Layers**

1. **Controller Layer**
  - Handles API requests
  - Maps endpoints
2. **Service Layer**
  - Contains business logic
  - Processes data
3. **Repository Layer**
  - Interacts with MongoDB
  - Performs database queries

## **B. API Workflow**

1. Receive request from frontend
2. Process request in service layer
3. Fetch data from database
4. Send response to frontend

## **C. Advantages**

- Scalable architecture
- Clean separation of concerns
- Efficient data processing

## **4.1.5 Database Implementation (MongoDB)**

### **A. Collections Used**

1. **Hospital Collection**
  - Stores hospital details
2. **Doctor Collection**
  - Stores doctor information

### **B. Data Format**

MongoDB stores data in JSON-like format, allowing flexible schema design.

### **C. Operations**

- Insert hospital and doctor data
- Retrieve data using queries
- Update records if needed

## **4.1.6 API Integration Workflow**

### **A. Steps**

1. User performs action (click/filter)
2. React sends API request
3. Spring Boot processes request
4. MongoDB returns data
5. Backend sends response

6. Frontend displays data

## **B. Tools Used**

- fetch / axios (frontend)
- REST APIs (backend)

### **4.1.7 User Interface Implementation**

The user interface is developed using **React**, ensuring a dynamic and responsive design.

#### **Features:**

- Hospital card layout
- Doctor profile cards
- Department filter dropdown
- Smooth navigation
- Responsive design

#### **Components Used:**

- HospitalList
- DoctorList
- DashboardPage

### **4.1.8 Admin Login Module Implementation**

#### **A. Purpose**

The Admin Login module is implemented to provide secure access for administrators to manage the system. It ensures that only authorized users can access administrative functionalities.

#### **B. Features**

- Secure login authentication using username and password
- Restricted access to dashboard and system data
- Separate interface for admin operations

#### **C. Workflow**

1. Admin enters login credentials
2. Credentials are validated (frontend/backend)
3. If valid → Redirect to dashboard
4. If invalid → Show error message

#### **D. Technologies Used**

- React (Login UI)
- JavaScript (Validation logic)

- Spring Boot (optional backend authentication)

## E. Outputs

- Successful login → Dashboard access
- Invalid login → Error notification

## 4.2 Testing / Verification Plan

A systematic testing process ensures that all modules function correctly and efficiently.

### Functional Test Cases

| Test ID | Component        | Test Case                   | Expected Output               | Result |
|---------|------------------|-----------------------------|-------------------------------|--------|
| T01     | Hospital Display | Load hospitals              | Hospitals displayed correctly | Pass   |
| T02     | Doctor Listing   | Select hospital             | Doctors list shown            | Pass   |
| T03     | Filtering        | Apply department filter     | Filtered doctors displayed    | Pass   |
| T04     | API Response     | Fetch data                  | Correct data returned         | Pass   |
| T05     | UI Navigation    | Switch pages                | Smooth navigation             | Pass   |
| T06     | Data Integration | Backend-frontend connection | Data displayed correctly      | Pass   |

## 4.3 Result Analysis / Screenshots

The results of the Medical Tourism Platform confirm that the system works efficiently and meets all functional requirements. The following screenshots demonstrate the successful implementation of different modules of the system.

### 4.3.1 Login Page

The login page serves as the entry point of the application. It allows users to securely access the system by entering valid credentials.

**Features:**

- User authentication system
- Input fields for username and password
- Validation for correct login details

**Result:**

- Users are able to log in successfully
- Invalid credentials are handled properly
- Ensures secure access to the dashboard

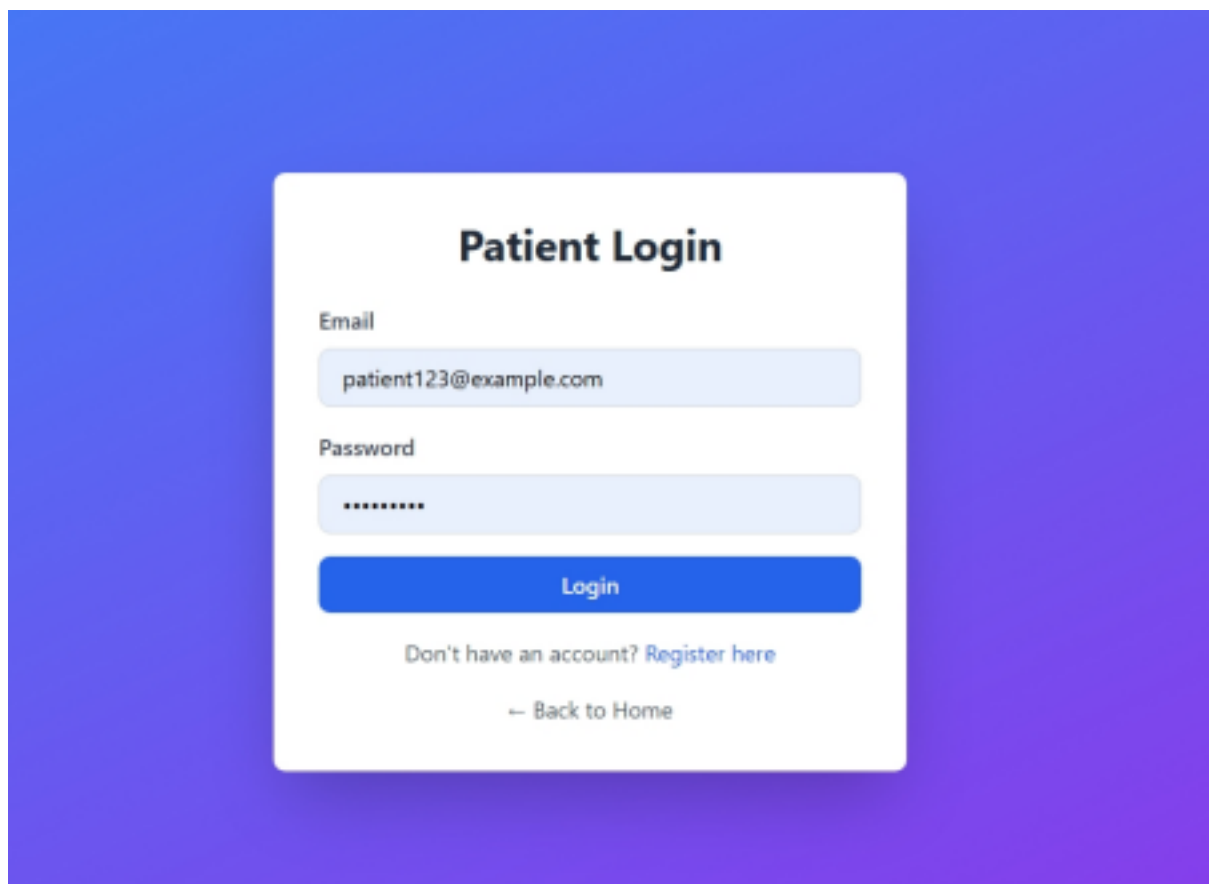


Figure 1: Login Page

### 4.3.2 Dashboard Page

The dashboard is the main interface of the application, where users can view available hospitals and navigate through the system.

**Features:**

- Display of hospitals in card format
- Easy navigation options
- User-friendly layout

**Result:**

- Hospitals are displayed clearly with proper structure
- Users can easily select a hospital
- Smooth navigation is achieved

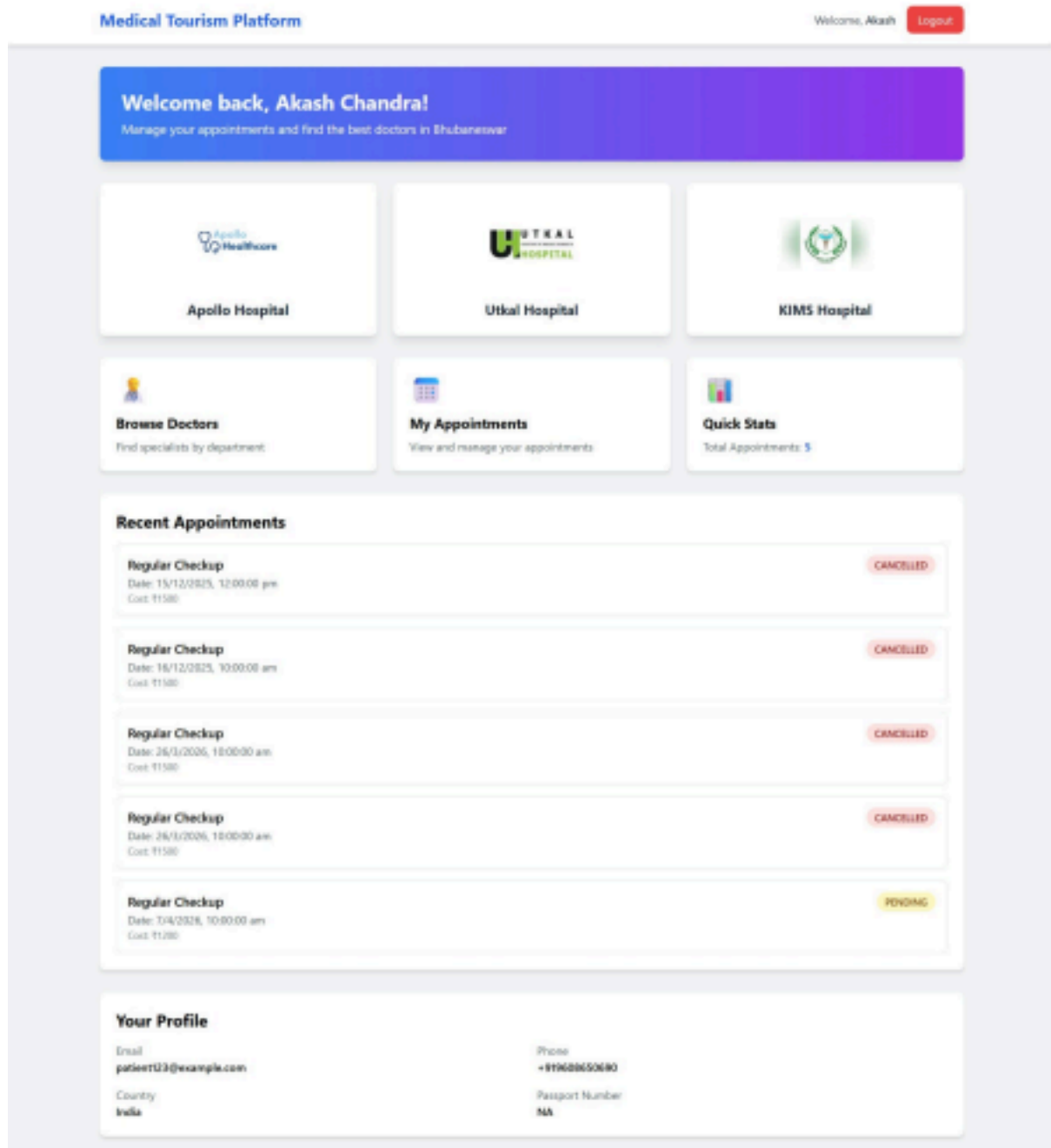


Figure 2: Dashboard Page

### 4.3.3 Doctor List Page

This page displays the list of doctors associated with the selected hospital.

#### Features:

- Doctor profile cards
- Details such as specialization, experience, and consultation fee
- Dynamic data loading from backend

#### Result:

- Doctors are displayed correctly for each hospital
- Information is accurate and well-organized
- UI is responsive and easy to understand

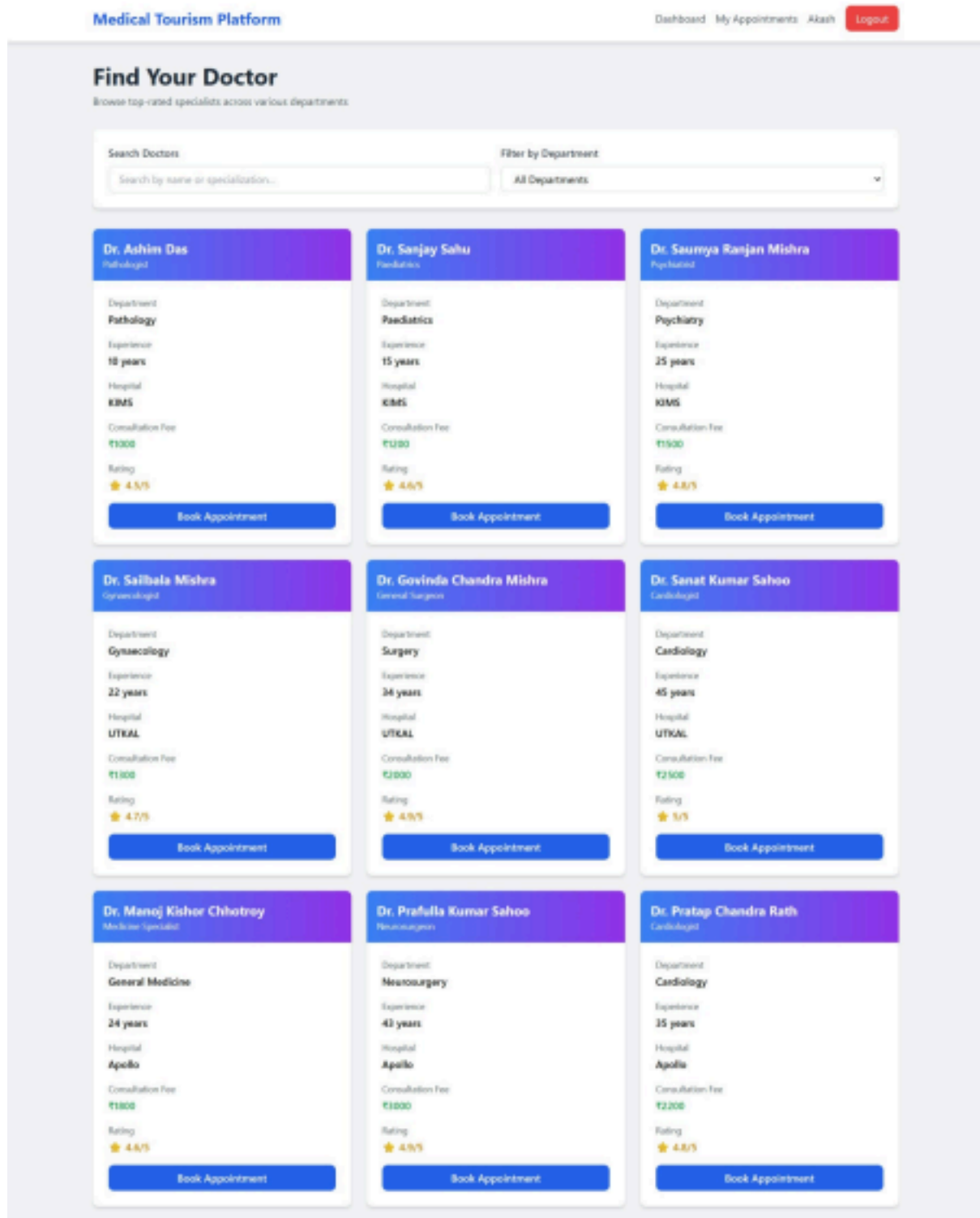


Figure 3: Doctor List Page

### 4.3.4 Appointment List Page

The appointment page shows the list of appointments booked by users.

#### Features:

- Displays appointment details
- Organized list format
- Easy tracking of bookings

#### Result:

- Appointment data is displayed correctly
- Users can view their bookings easily
- System ensures proper data handling

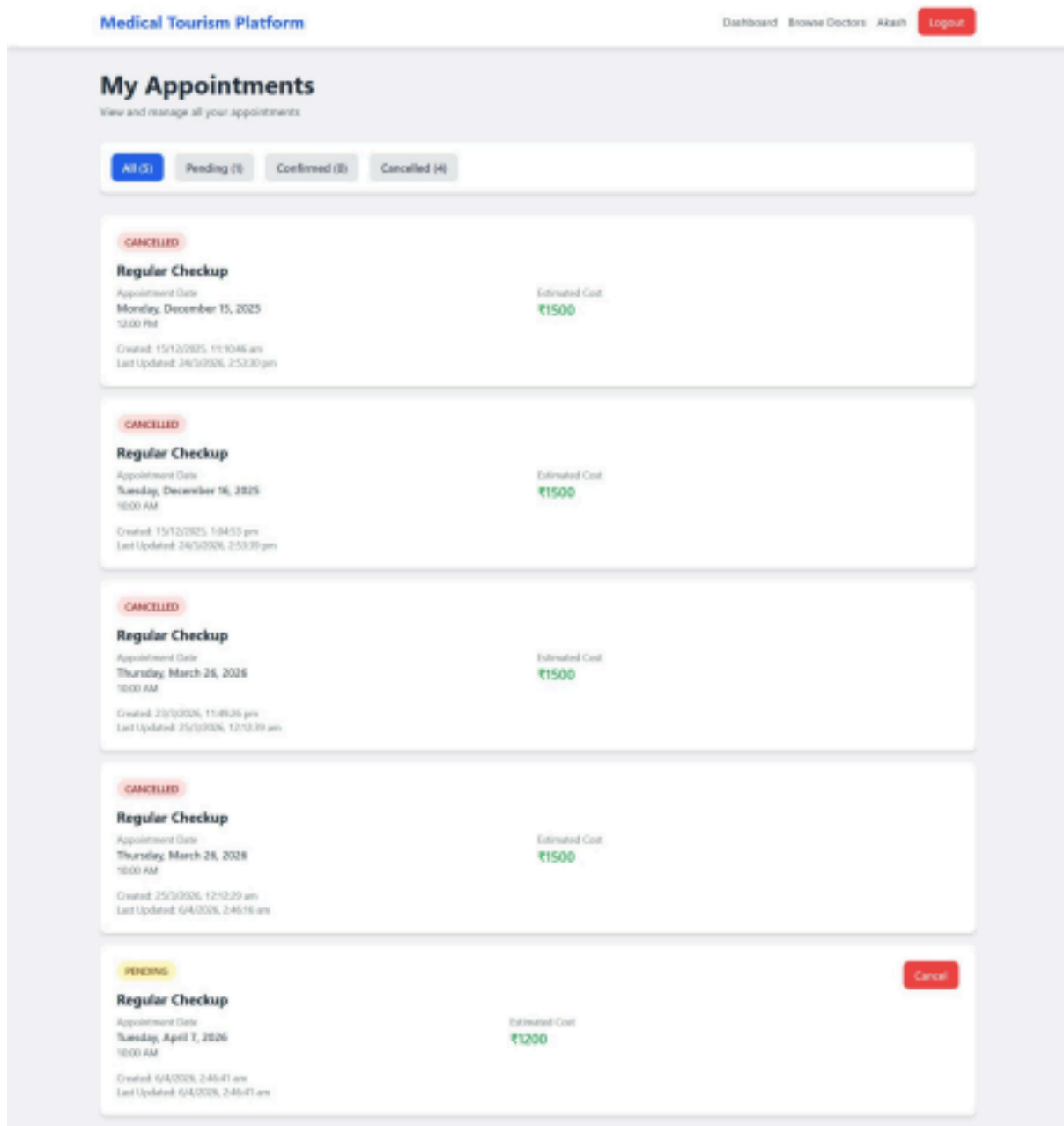


Figure 4: Appointment List Page

### **4.3.5 Admin Page**

The Admin Page is designed to provide secure access to authorized users for managing the system.

- The admin can log in using valid credentials (username and password)
- Unauthorized users cannot access the system
- Displays error message for invalid login attempts
- After successful login, admin is redirected to the dashboard

This module ensures security and controlled access to the system.

## **4.4 Quality Assurance**

To ensure the reliability, performance, and usability of the Medical Tourism Platform, several quality assurance (QA) practices were adopted during development and testing phases.

### **1. Unit Testing**

Each module of the system was tested independently to verify its correct functionality.

- Testing of React components (HospitalList, DoctorList, DashboardPage)
- Validation of API endpoints in Spring Boot
- Verification of data retrieval from MongoDB
- Testing filtering logic for department-based search

### **2. Integration Testing**

Integration testing was performed to ensure smooth communication between different system components.

- Frontend (React) and Backend (Spring Boot) integration
- Backend and MongoDB database connectivity
- API response handling and data rendering in UI
- End-to-end data flow from user action to output display

### **3. Performance Evaluation**

The system was evaluated for speed and efficiency to ensure a smooth user experience.

- Fast API response time for fetching hospital and doctor data

- Smooth UI rendering without delays
- Efficient filtering with minimal latency
- Stable performance under multiple user interactions

#### **4. Security Assessment**

Basic security measures were implemented to ensure safe usage of the system.

- Validation of user inputs to prevent invalid data
- Secure handling of login credentials
- Controlled API access
- No exposure of sensitive backend data

#### **5. User Acceptance Testing**

The system was tested by users to evaluate usability and functionality.

- Users tested login, dashboard, and navigation
- Verified doctor listing and filtering features
- Evaluated ease of use and interface clarity
- Feedback confirmed smooth and user-friendly experience

The quality assurance process ensured that the Medical Tourism Platform is reliable, efficient, secure, and user-friendly, meeting all functional and performance requirements.

# CHAPTER 5

## STANDARDS ADOPTED

To ensure quality, consistency, and reliability in the Medical Dashboard project, clear standards were followed during design, coding, and testing.

### 5.1 Design Standards

The system was designed using modular architecture principles, allowing independent development and maintenance of components. UML diagrams were employed to visualize workflows and data flow, ensuring clarity in communication and planning. The design emphasised scalability and usability, making future enhancements easier to integrate.

### 5.2 Coding Standards

Coding practices adhered to industry norms for readability and maintainability. In the frontend, React components were developed with modularity and separation of concerns. In the backend, Spring Boot services followed a layered architecture with controllers, services, and repositories. Consistent naming conventions, indentation, and documentation were maintained throughout the project.

### 5.3 Testing Standards

Testing was carried out systematically to validate functionality and performance. APIs were verified using Postman to ensure accurate data retrieval. Unit tests confirmed backend logic, while manual testing of the frontend checked navigation, responsiveness, and department filtering. The testing process emphasized reliability, user experience, and error handling.

In summary, the adoption of design, coding, and testing standards ensured that the Medical Dashboard was developed in a structured and professional manner.

# CHAPTER 6

## CONCLUSION AND FUTURE SCOPE

### 6.1 Conclusion

The project successfully implements a hospital-wise doctor search dashboard with department-based filtering. By integrating a Spring Boot backend, MongoDB database, and React frontend, the system provides a structured and user-friendly platform for patients to locate doctors across different hospitals. The dashboard displays hospital cards with logos, doctor profiles with key details, and supports filtering by specialisation. Testing confirmed that hospital navigation, doctor listing, and department search work reliably, validating the effectiveness of the system. Overall, the project demonstrates how modern software engineering practices can be applied to solve real-world healthcare challenges.

#### **Key Achievements of the System:**

##### **1. Hospital-Wise Doctor Search**

The platform allows users to explore hospitals and view doctors associated with each hospital. This structured approach eliminates the problem of scattered information and helps users quickly locate relevant healthcare providers.

##### **2. Department-Based Filtering**

The system enables users to filter doctors based on specialization such as cardiology, neurology, and surgery. This improves search efficiency and helps users identify specialists easily.

##### **3. Interactive and User-Friendly Interface**

The use of React ensures a responsive and interactive user interface. Features such as hospital cards, doctor profiles, and smooth navigation enhance the overall user experience.

##### **4. Efficient Backend and Data Handling**

The Spring Boot backend provides RESTful APIs for seamless communication between frontend and database. MongoDB ensures flexible and efficient storage of hospital and doctor data.

##### **5. Appointment and Navigation System**

The platform includes features such as login, dashboard navigation, doctor listing, and appointment viewing, making it a complete and practical healthcare solution.

##### **6. Secure Admin Login System**

The system includes an admin login module that ensures secure and controlled access to the platform. Only authorized administrators can log in and access system functionalities, improving security, reliability, and system management.

## 6.2 Future Scope

While the current system meets its core objectives, several enhancements can be introduced to extend its functionality:

- **Appointment Booking:** Allow patients to schedule consultations directly through the dashboard.
- **Payment Gateway Integration:** Enable secure online payments for appointments and medical services.
- **Patient Reviews and Ratings:** Provide feedback mechanisms to improve transparency and trust.
- **Advanced Search Features:** Incorporate filters for experience, consultation fees, and availability.
- **Mobile Application:** Extend the dashboard to mobile platforms for wider accessibility.

In conclusion, the Medical Dashboard lays a strong foundation for digital healthcare solutions. With future enhancements such as booking, payments, and patient feedback, the system can evolve into a comprehensive healthcare platform that benefits both patients and providers.

# References

1. React Documentation. (2024). *React – A JavaScript library for building user interfaces*.  
<https://react.dev>
2. Spring Boot Documentation. (2024). *Spring Boot – Building production-ready applications*.  
<https://spring.io/projects/spring-boot>
3. MongoDB Documentation. (2024). *MongoDB – NoSQL Database Documentation*.  
<https://www.mongodb.com/docs>
4. Mozilla Developer Network (MDN). (2024). *HTML, CSS, and JavaScript Documentation*.  
<https://developer.mozilla.org>
5. Oracle. (2024). *Java Documentation*.  
<https://docs.oracle.com>
6. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures (REST APIs)*. University of California, Irvine.
7. Sommerville, I. (2016). *Software Engineering (10th Edition)*. Pearson.
8. IEEE. (2020). *IEEE Standard for Software Engineering*.
9. OpenAPI Initiative. (2024). *API Design and Documentation Standards*.  
<https://www.openapis.org>

# INDIVIDUAL CONTRIBUTION REPORT:

## AKASH CHANDRA (22051659)

### Abstract

The Medical Tourism Platform is a web-based healthcare system designed to provide structured information about hospitals and doctors. My primary contribution focused on **complete Backend Development using Spring Boot**, along with the implementation of a **secure Admin Login System**. I was responsible for developing RESTful APIs, handling business logic, managing database interactions, and ensuring secure authentication. This contribution forms the backbone of the system, enabling smooth communication between frontend and database while maintaining system security.

## Individual Contribution and Findings

### 1. Backend Development (Spring Boot)

I developed the complete backend of the application using Spring Boot.

- Designed and implemented **Controller, Service, and Repository layers**
- Managed request-response cycle between frontend and backend
- Ensured modular and scalable backend architecture
- Implemented business logic for hospital and doctor data

### 2. RESTful API Development

I created APIs to handle data operations and communication.

- Developed APIs for:
  - Fetching hospital data
  - Fetching doctor data
  - Filtering doctors by hospital
- Ensured proper request handling and response formatting
- Tested APIs for accuracy and performance

### 3. Database Integration (MongoDB)

I connected the backend with MongoDB for data storage.

- Designed collections for hospitals and doctors
- Performed CRUD operations (Create, Read, Update, Delete)
- Ensured efficient data retrieval and storage

### 4. Admin Login System Implementation

I implemented a secure admin authentication system.

- Developed login API for authentication

- Validated username and password
- Restricted access to authorized users only
- Handled invalid login attempts with proper error messages

This feature ensures secure system access and controlled management.

## 5. API Integration and Data Flow

I ensured smooth communication between the frontend and the backend.

- Processed API requests from frontend
- Retrieved data from database
- Sent structured responses back to frontend
- Handled errors and exceptions efficiently

### Key Findings During Development:

During backend development, I observed the following:

- Layered architecture improves maintainability and scalability
- Proper API design ensures smooth frontend-backend communication
- Secure authentication is essential for protecting system access
- Efficient database queries improve performance

Full Signature of Supervisor:

.....

Full signature of the student:

.....

# INDIVIDUAL CONTRIBUTION REPORT:

## KAUSHAL KUMAR (22051687)

### Abstract

The Medical Tourism Platform requires efficient data storage and retrieval for managing hospital and doctor information. My primary contribution focused on Database Design and Management using MongoDB, ensuring structured, scalable, and efficient handling of healthcare data. I was responsible for designing the database schema, managing data operations, and optimizing queries to support fast and reliable system performance.

## Individual Contribution and Findings

### 1. Database Design

I designed the database structure to store hospital and doctor information efficiently.

- Created separate collections for hospitals and doctors
- Defined schema structure with fields such as:
  - Hospital name, location, and details
  - Doctor name, specialization, experience, fees and ensured proper relationship between hospital and doctor data

### 2. Data Management

I handled the storage and organization of data in MongoDB.

- Inserted hospital and doctor records into database
- Maintained structured and clean data
- Ensured data consistency and integrity
- Updated records when required

### 3. Query Optimization

I optimized database queries to improve system performance.

- Implemented efficient queries for fetching data
- Reduced data retrieval time

## Key Findings During Development

During database development, I observed the following:

- NoSQL databases like MongoDB provide flexibility for dynamic data
- Proper schema design is essential for efficient data management

Full Signature of Supervisor:

.....

Full signature of the student:

.....

# INDIVIDUAL CONTRIBUTION REPORT:

**SHILKY DWIVEDI (22052058)**

## **Abstract:**

The Medical Tourism Platform aims to provide a user-friendly interface for accessing hospital and doctor information efficiently. My primary contribution focused on **Frontend Development and User Interface Design** using React, HTML, CSS, and JavaScript. I was responsible for designing and implementing interactive pages such as login, dashboard, doctor listing, and appointment management. My work ensures a smooth user experience, responsive design, and effective visualization of data fetched from backend APIs.

## **Individual Contribution and Findings**

### **1. Frontend Development (React)**

I developed the user interface of the application using React.

- Built reusable components such as:
  - Login Page
  - Dashboard Page
  - Doctor List Page
  - Appointment List Page
- Implemented component-based architecture for better code structure
- Ensured smooth navigation between pages

### **2. User Interface Design**

I designed a clean and user-friendly interface for better usability.

- Used HTML and CSS for layout and styling
- Maintained consistent design across all pages
- Created responsive layouts for different screen sizes
- Focused on simplicity and easy navigation

### **3. API Integration**

I integrated frontend with backend APIs to display dynamic data.

- Used fetch/axios to call APIs
- Retrieved hospital and doctor data from backend
- Displayed real-time data on UI
- Handled API responses and error cases

#### 4. Doctor Listing and UI Rendering

I implemented the display of doctor information on the frontend.

- Displayed doctor details in structured card format
- Rendered dynamic data based on selected hospital
- Ensured proper alignment and readability

#### 5. Appointment Page Implementation

I developed the appointment list page for displaying booking details.

- Displayed appointment records clearly
- Organized data in a structured format
- Improved user experience for viewing appointments

### Key Findings During Development

During frontend development, I observed the following:

- A clean and simple UI greatly improves user experience
- React's component-based approach enhances code reusability
- Proper API integration enables real-time data updates
- Responsive design is essential for accessibility across devices

Full Signature of Supervisor:

.....

Full signature of the student:

.....

# INDIVIDUAL CONTRIBUTION REPORT:

## NAMITHA MOHAN (2206415)

### Abstract

The Medical Tourism Platform is designed to provide users with an efficient way to search and access healthcare services. My primary contribution focused on the development of the **Department-Based Filtering Module**, which allows users to find doctors based on their specialisation, and the **preparation of the project documentation**. These contributions play an important role in improving the usability of the system and ensuring a clear presentation of the project work.

## Individual Contribution and Findings

### 1. Department-Based Filtering Module

I developed the functionality that allows users to filter doctors based on their department or specialization.

- Implemented filtering options for various departments such as:
  - Cardiology
  - Neurology
  - Orthopedics
  - General Medicine
- Ensured that when a department is selected, only relevant doctors are displayed
- Reduced unnecessary data shown to users, making the interface cleaner
- Improved search efficiency and user experience

### 2. Filtering Logic and Data Handling

I worked on implementing the logic required for filtering and updating the UI dynamically.

- Applied filtering conditions using JavaScript
- Managed selected department state
- Ensured real-time updates without page reload
- Integrated filtered data with the doctor listing component

### 3. User Interface Support for Filtering

I contributed to enhancing the UI for better usability of filtering features.

- Designed simple and easy-to-use filter options
- Ensured proper placement of filter controls
- Maintained consistency with overall UI design
- Improved readability of filtered results

### 4. Project Report Preparation

I actively contributed to the preparation and structuring of the project report.

- Assisted in writing different sections of the report such as:
  - Introduction
  - System Design
  - Implementation
  - Conclusion
- Ensured proper formatting and alignment of content
- Maintained clarity, consistency, and proper structure
- Helped in organizing diagrams, headings, and content flow

## **Key Findings During Development**

During my contribution, I observed the following:

- Department-based filtering significantly improves search efficiency
- Clean and simple UI design enhances user interaction
- Dynamic data updates improve overall system performance
- Proper documentation is essential for explaining project functionality clearly

Full Signature of Supervisor:

.....

Full signature of the student:

.....

# INDIVIDUAL CONTRIBUTION REPORT:

## Amisha Tarun Dewangan(2229204)

### Abstract

The Medical Tourism Platform is a web-based healthcare system designed to simplify the process of finding hospitals and doctors. My primary contribution focused on **preparing the project presentation (PPT)** and organizing the project content in a clear and visually effective manner. The presentation plays an important role in explaining the project workflow, features, and technical aspects in a structured and easy-to-understand format.

## Individual Contribution and Findings

### 1. PPT Design and Structure

I designed and structured the complete project presentation.

- Created slides for all major sections such as:
  - Introduction
  - Problem Statement
  - System Architecture
  - Implementation
- Maintained proper sequence and logical flow of content
- Ensured clarity and readability of information

### 2. Visual Design and Formatting

I focused on making the presentation visually appealing.

- Applied consistent theme, colors, and fonts
- Added diagrams, flowcharts, and screenshots

### 3. Presentation Enhancement

I improved the quality and effectiveness of the presentation.

- Reduced unnecessary content for better clarity
- Highlighted key points for quick understanding
- Structured slides for smooth explanation during viva
- Ensured the presentation is easy to present and explain

## Key Findings During Development

During the preparation of the presentation, I observed the following:

- A well-structured PPT improves understanding of the project
- Visual elements make concepts easier to explain

- Short and clear points are more effective than long text
- Proper formatting enhances presentation quality

Full Signature of Supervisor:

.....

Full signature of the student:

.....

# INDIVIDUAL CONTRIBUTION REPORT:

**PRANJAL MOHANTY (2229137)**

## Abstract

My role focused on **System Integration and Deployment Support**, ensuring all components work together smoothly.

## Individual Contribution and Findings

### 1. System Integration

- Connected frontend with backend APIs
- Ensured proper data flow between database and UI
- Checked that all modules are working together correctly

### 2. Testing of System

- Tested login, dashboard, doctor list, and appointment pages
- Verified filtering and navigation features
- Ensured all functionalities are working properly

### 3. Error Handling

- Identified errors during integration
- Helped in fixing bugs
- Checked correct API responses

### 4. Deployment Support

- Helped in final project setup
- Ensured project runs properly on system
- Supported team during final execution

## Key Findings

- Integration is important for overall system performance
- Testing helps in identifying errors
- Proper setup ensures smooth execution

Full Signature of Supervisor:

.....

Full signature of the student:

.....

